人工智能 – Project1

DL实践 - 街景字符识别

复旦大学计算与智能创新学院

陈智能

2026/4/24

任务介绍

当一个无人快递车第一次来到一个复杂的街区，如何识别一个门牌号呢?

赛题提供了街景门牌数据集，要求选手针对这一场景设计解决方案，实现对门牌号的正确预测，是一个典型的字符识别问题。

https://tianchi.aliyun.com/competition/entrance/531795

数据集介绍

赛题来源自Google街景图像中的门牌号数据集（The Street View House Numbers Dataset, SVHN），并根据一定方式采样得到比赛数据集。

训练集数据包括3W张照片，验证集数据包括1W张照片，每张照片包括RGB图像和对应的编码以及具体位置；测试集包括4W张照片，只有RGB图像，不提供编码与各字符的具体位置。

Field	Description
top	左上角坐标Y
height	字符高度
left	左上角坐标X
width	字符宽度
label	字符编码

"000001.png": {"height": [32, 32], "label": [2, 3], "left": [77, 98], "top": [29, 25], "width": [23, 26]}

矩形检测框常用的两种表示方式：

1. 左上角坐标+检测框高度+检测框宽度
2. 左上角坐标+右下角坐标

Baseline

1. 下载完整数据集

```
# 下载数据集
links = pd.read_csv('./mchar_data_list_0515.csv')  #修改成你电脑对应的路径
dir_name = 'NDataset'
mypath = './crnn/'
dataset_path = os.path.join(mypath,dir_name)
print(f"数据集目录: {dataset_path}")
if not os.path.exists(dataset_path):
    os.mkdir(dataset_path)
for i,link in enumerate(links['link']):
    file_name = links['file'][i]
    print(file_name, '\t', link)
    file_name = os.path.join(mypath,dir_name,file_name)
    if not os.path.exists(file_name):
        response = requests.get(link, stream=True)
        with open(file_name, 'wb') as f:
            for chunk in response.iter_content(chunk_size=1024):
                if chunk:
                    f.write(chunk)
zip_list = ['mchar_train', 'mchar_test_a', 'mchar_val']
for little_zip in zip_list:
    zip_name = os.path.join(mypath,dir_name,little_zip)
    if not os.path.exists(zip_name):
        zip_file = zipfile.ZipFile(os.path.join(mypath,dir_name,f"{little_zip}.zip"), 'r')
        zip_file.extractall(path = mypath + dir_name)
if os.path.exists(os.path.join(mypath,dir_name,'__MACOSX')):
    shutil.rmtree(os.path.join(mypath,dir_name,'__MACOSX'))
```

从elearning或官网下载

下载完整数据集文件压缩包

解压得到数据集文件夹

Baseline

2. 对训练数据进行统计

```
1个数字的图片数目： 4636
2个数字的图片数目： 16262
3个数字的图片数目： 7813
4个数字的图片数目： 1280
5个数字的图片数目： 8
6个数字的图片数目： 1
```

有限长度的字符识别可转化为对每个数位做0-9与”空“的分类任务，将每个数位合并即得到最后的识别结果。考虑数据长度分布特点，只用四个分类头对应识别前四个位置上的数字，没有用到提供的检测框数据。

Baseline

3. 构建数据集

```python
class DigitsDataset(Dataset):
    """
    DigitsDataset
    Params:
      data_dir(string): data directory
      label_path(string): label path
      aug(bool): wheather do image augmentation, default: True
    """
    def __init__(self, mode='train', size=(128, 256), aug=True):
        super(DigitsDataset, self).__init__()
        self.aug = aug
        self.size = size
        self.mode = mode
        self.width = 224
        self.batch_count = 0
        if mode == 'test':
            self.imgs = glob(data_dir['test_data'] + '*.png')
            self.labels = None
        else:
            labels = json.load(open(data_dir['%s_label' % mode], 'r'))
            imgs = glob(data_dir['%s_data' % mode] + '*.png')
            self.imgs = [(img, labels[os.path.split(img)[-1]]) for img in imgs \
                         if os.path.split(img)[-1] in labels]
```

测试集无label信息

遍历构建训练集与验证集

Baseline

3. 构建数据集

```python
def __getitem__(self, idx):
    if self.mode != 'test':
        img, label = self.imgs[idx]
    else:
        img = self.imgs[idx]
        label = None
    img = Image.open(img)
    trans0 = [
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ]
    min_size = self.size[0] if (img.size[1] / self.size[0]) < ((img.size[0] / self.size[1])) else self.size[1]
    trans1 = [
        transforms.Resize(128),
        transforms.CenterCrop((128, self.width))
    ]
    if self.aug:
        trans1.extend([
            transforms.ColorJitter(0.1, 0.1, 0.1),
            transforms.RandomGrayscale(0.1),
            transforms.RandomAffine(15, translate=(0.05, 0.1), shear=5)
        ])
    trans1.extend(trans0)
    if self.mode != 'test':
        return transforms.Compose(trans1)(img), t.tensor(
            label['label'][:4] + (4 - len(label['label'])) * [10]).long()
    else:
        # trans1.append(transforms.RandomErasing(scale=(0.02, 0.1)))
        return transforms.Compose(trans1)(img), self.imgs[idx]
```

简单的数据增强

将空的label处理为10

```python
def __len__(self):
    return len(self.imgs)

def collect_fn(self, batch):
    imgs, labels = zip(*batch)
    if self.mode == 'train':
        if self.batch_count > 0 and self.batch_count % 10 == 0:
            self.width = random.choice(range(224, 256, 16))

    self.batch_count += 1
    return t.stack(imgs).float(), t.stack(labels)
```

Baseline

4. 定义模型

```python
class DigitsResnet50(nn.Module):
    def __init__(self, class_num=11):
        super(DigitsResnet50, self).__init__()
        self.net = resnet50(pretrained=True)
        self.net = nn.Sequential(*list(self.net.children())[:-1])
        self.cnn = self.net
        self.fc1 = nn.Linear(2048, class_num)
        self.fc2 = nn.Linear(2048, class_num)
        self.fc3 = nn.Linear(2048, class_num)
        self.fc4 = nn.Linear(2048, class_num)

    def forward(self, img):
        feat = self.cnn(img)
        feat = feat.view(feat.shape[0], -1)
        c1 = self.fc1(feat)
        c2 = self.fc2(feat)
        c3 = self.fc3(feat)
        c4 = self.fc4(feat)
        return c1, c2, c3, c4
```

预训练的Resnet50
去掉原模型最后的linear层

提取特征后，
做前四个位置字符的分类

Baseline

5. 定义损失函数

```python
class LabelSmoothEntropy(nn.Module):
    def __init__(self, smooth=0.1, class_weights=None, size_average='mean'):
        super(LabelSmoothEntropy, self).__init__()
        self.size_average = size_average
        self.smooth = smooth
        self.class_weights = class_weights

    def forward(self, preds, targets):
        lb_pos, lb_neg = 1 - self.smooth, self.smooth / (preds.shape[0] - 1)
        smoothed_lb = t.zeros_like(preds).fill_(lb_neg).scatter_(1, targets[:, None], lb_pos)
        log_soft = F.log_softmax(preds, dim=1)
        if self.class_weights is not None:
            loss = -log_soft * smoothed_lb * self.class_weights[None, :]
        else:
            loss = -log_soft * smoothed_lb
        loss = loss.sum(1)
        if self.size_average == 'mean':
            return loss.mean()
        elif self.size_average == 'sum':
            return loss.sum()
        else:
            raise NotImplementedError
```

基于交叉熵损失函数

对标签进行平滑操作

如原先gt标签为[1,0,0]，现在根据smooth调整变为[0.9,0.05,0.05]

优点：

1. 提升模型的泛化能力
2. 防止模型过度自信，缓解过拟合
3. 对标签噪声有一定的抗干扰能力

Baseline

6. 训练代码

```python
def train_epoch(self, epoch):
    total_loss = 0
    corrects = 0
    tbar = tqdm(self.train_loader)
    self.model.train()
    for i, (img, label) in enumerate(tbar):
        # print(img.shape)
        img = img.to(self.device)
        label = label.to(self.device)
        self.optimizer.zero_grad()
        pred = self.model(img)
        loss = self.criterion(pred[0], label[:, 0]) + \
               self.criterion(pred[1], label[:, 1]) + \
               self.criterion(pred[2], label[:, 2]) + \
               self.criterion(pred[3], label[:, 3])
        total_loss += loss.item()
        loss.backward()
        self.optimizer.step()
        temp = t.stack([ \
            pred[0].argmax(1) == label[:, 0], \
            pred[1].argmax(1) == label[:, 1], \
            pred[2].argmax(1) == label[:, 2], \
            pred[3].argmax(1) == label[:, 3], ], dim=1)
        corrects += t.all(temp, dim=1).sum().item()
        tbar.set_description(
            'loss: %.3f, acc: %.3f' % (loss / (i + 1), corrects * 100 / ((i + 1) * config.batch_size)))
        if (i + 1) % config.print_interval == 0:
            self.lr_scheduler.step()
    return corrects * 100 / ((i + 1) * config.batch_size)
```

设置为训练模式

梯度清零

计算损失

反向传播

更新参数

Baseline

7. 验证代码

```python
def eval(self):
    self.model.eval()
    corrects = 0
    with t.no_grad():
        tbar = tqdm(self.val_loader)
        for i, (img, label) in enumerate(tbar):
            img = img.to(self.device)
            label = label.to(self.device)
            pred = self.model(img)
            temp = t.stack([
                pred[0].argmax(1) == label[:, 0], \
                pred[1].argmax(1) == label[:, 1], \
                pred[2].argmax(1) == label[:, 2], \
                pred[3].argmax(1) == label[:, 3], \
                ], dim=1)
            corrects += t.all(temp, dim=1).sum().item()
            tbar.set_description('Val Acc: %.2f' % (corrects * 100 / ((i + 1) * config.batch_size)))
    self.model.train()
    return corrects / (len(self.val_loader) * config.batch_size)
```

Baseline

8. 评测代码

```python
def predicts(model_path):
    test_loader = DataLoader(DigitsDataset(mode='test', aug=False), batch_size=config.batch_size, shuffle=False,\
                             num_workers=8, pin_memory=True, drop_last=False)
    results = []
    res_path = model_path
    res_net = DigitsResnet50().cuda()
    res_net.load_state_dict(t.load(res_path)['model'])
    print('Load model from %s successfully'%model_path)
    tbar = tqdm(test_loader)
    res_net.eval()
    with t.no_grad():
        for i, (img, img_names) in enumerate(tbar):
            img = img.cuda()
            pred = res_net(img)
            results += [[name, code] for name, code in zip(img_names, parse2class(pred))]
    results = sorted(results, key=lambda x: x[0])
    write2csv(results)
    return results
```

```python
def parse2class(prediction):
    """
    Params:
    prediction(tuple of tensor):
    """
    ch1, ch2, ch3, ch4 = prediction
    char_list = [str(i) for i in range(10)]
    char_list.append('')
    ch1, ch2, ch3, ch4 = ch1.argmax(1), ch2.argmax(1), ch3.argmax(1), ch4.argmax(1)
    ch1, ch2, ch3, ch4 = [char_list[i.item()] for i in ch1], [char_list[i.item()] for i in ch2], \
                         [char_list[i.item()] for i in ch3], [char_list[i.item()] for i in ch4]
    res = [c1+c2+c3+c4 for c1, c2, c3, c4 in zip(ch1, ch2, ch3, ch4)]
    return res
```

```python
def write2csv(results):
    """
    results(list):
    """
    # 定义输出文件
    df = pd.DataFrame(results, columns=['file_name', 'file_code'])
    df['file_name'] = df['file_name'].apply(lambda x: x.split('/')[-1])
    save_name = 'results-mb.csv'
    df.to_csv(save_name, sep=',', index=None)
    print('Results.saved to %s'%save_name)
```

改进思路

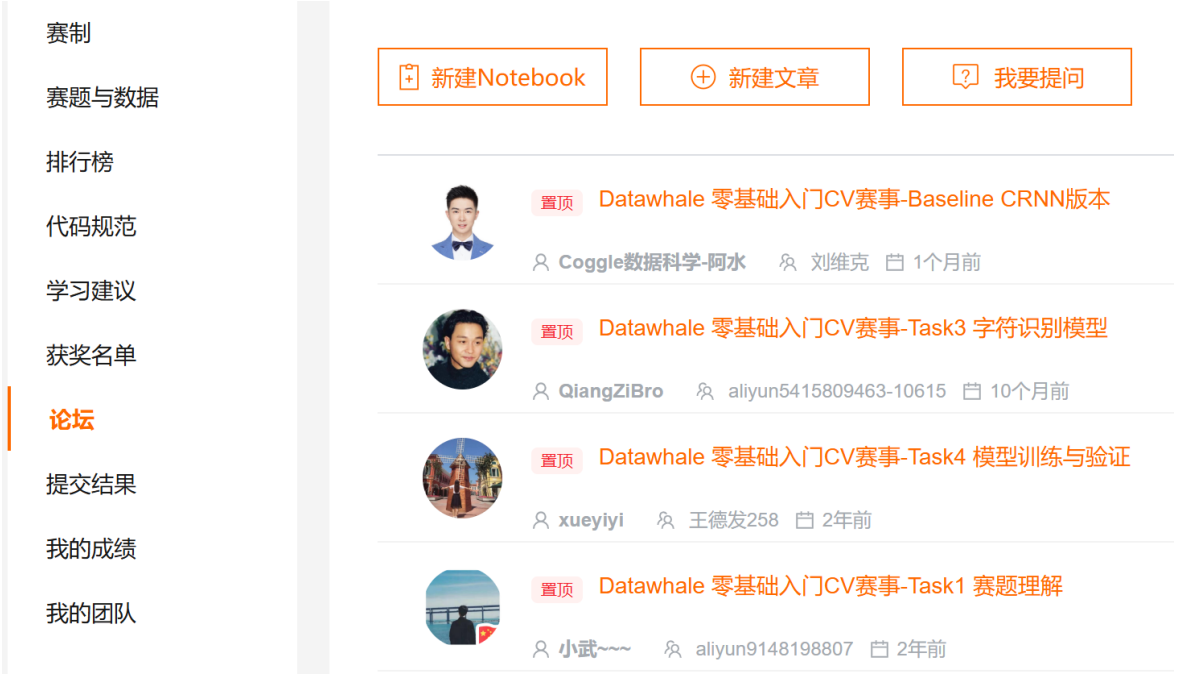
1. 在Baseline基础上改进，包括但不限于选用更深更先进的CNN网络或其它模型，调整数据增强方案，调整损失函数，调整超参数等；

2. 针对数据集全是数字，且每个数字都提供检测框数据的特点，将不同的数字作为不同的检测类处理，训练目标检测模型，如yolo系列（https://github.com/ultralytics/ultralytics）；

3. 将数字视为常规场景文本，训练文本检测模型以提供更精准的文本实例框，训练文本识别模型来更精准识别，不一定要从头训练，可以在预训练模型上微调；

4. 模型集成方案，综合考虑不同模型的预测结果，对概率分布或最终预测结果等进行加权投票；

参考资源



论坛里有一些高分实现方案可以作为直接参考，直接复现就能得到不错的结果。若参考了论坛里的实现，要求必须在报告中引用链接，并要求在复现其结果的基础上有一定程度的修改（变好或变差都行）

https://github.com/Topdu/OpenOCR?tab=readme-ov-file#scene-text-recognition

https://paddlepaddle.github.io/PaddleOCR/latest/index.html

选择场景文本检测/识别任务中表现优异的，且有完整训练代码的模型，将数据处理成其要求格式，重新训练或微调模型

实验要求

1. 在报告中说明在**验证集**上的Acc指标，要求实验结果可复现 - 4分

   超过0.86得1分　超过0.88得2分　超过0.90得3分　超过0.92得4分

2. 报告中介绍实现方案，包括运行环境说明，模型设计，损失函数设计等 - 2分

3. 报告中介绍涨点所做的数据增强、调参等工作；介绍实现方案的创新性，或在开源方案基础上做的修改；介绍实验中遇到的困难及对应的解决方案 - 3分

提交完整实现代码（不含数据）+ 方案报告（附结果截图，上限4页）

至elearning，命名为“学号_姓名_课堂练习3.zip”　总分：8分

最后得分为：min(8, s1+s2+s3)　截止日期: 2025年5月8日23:59 !!

THANKS

复旦大学计算与智能创新学院

陈智能

2026/4/24